6_SQLite

September 21, 2026

Creado por:

Isabel Maniega

1 Bases de datos

SQlite

SQLite también es un gestor de bases de datos relacional pero con objetivos muy diferentes a MySQL, SQLServer, Oracle etc. Este gestor de base de datos tiene por objetivo ser parte de la misma aplicación con la que colabora, es decir no cumple los conceptos de cliente y servidor.

Para entender sus usos podemos dar algunos ejemplos donde se utiliza el gestor SQLite:

- Firefox usa SQLite para almacenar los favoritos, el historial, las cookies etc.
- También el navegador Opera usa SQLite.
- La aplicación de comunicaciones Skype de Microsoft utiliza SQLite
- Los sistemas operativos Android y iOS adoptan SQLite para permitir el almacenamiento y recuperación de datos.

SQLite es Open Source y se ha instalado por defecto con Python, es decir forma parte de la biblioteca estándar, no tenemos que instalar ningún módulo con pip.

Si nuestra aplicación necesita almacenar gran cantidad de información local con cierta estructura el empleo de SQLite es nuestra principal opción.

Creamos una tabla llamada Articulos donde tiene 3 columnas: - Código que será la clave primaria para relacionar con otras tablas, es de tipo entero. - Descripción de tipo texto - Precio de tipo número. articulos

codigo descripcion precio ———————— 1 boligrafo 1.2 2 libro 20 3 lapicero 0.85 Realizaremos el CRUB (Create Read Update Delete):

[1]:
```
# Importamos las librería sqlite3:
import sqlite3

# Creamos un archivo como extensión .db donde se almacenará la información, si␣
 ↪no existe se crea:
conexion=sqlite3.connect("database.db")
try:
    # Creamos la tabla anteriormente explicada:
```

```
    conexion.execute("""CREATE TABLE articulos (codigo integer primary key␣
 ↪autoincrement,
                        descripcion text, precio real)""")
    print("se creo la tabla articulos")
except sqlite3.OperationalError:
    # si ocurre un error al crear la tabla nos mostrará el siguiente error:
    print("La tabla articulos ya existe")
conexion.close()
```

```
se creo la tabla articulos
```

[2]:
```
# Crear:
# Insertamos información en la tabla creada anteriormente de articulos:

conexion=sqlite3.connect("database.db")
conexion.execute("INSERT INTO articulos(descripcion,precio) VALUES (?,?)",␣
 ↪("boligrafo", 1.2))
conexion.execute("INSERT INTO articulos(descripcion,precio) VALUES (?,?)",␣
 ↪("libro", 20))
conexion.execute("INSERT INTO articulos(descripcion,precio) VALUES (?,?)",␣
 ↪("lapicero", 0.85))
conexion.commit()
conexion.close()
```

[3]:
```
# Read
# Leer todos los datos de la tabla:

conexion = sqlite3.connect("database.db")
cursor = conexion.execute("SELECT codigo,descripcion,precio FROM articulos")
for fila in cursor:
    print(fila)
conexion.close()
```

```
(1, 'boligrafo', 1.2)
(2, 'libro', 20.0)
(3, 'lapicero', 0.85)
```

[4]:
```
# Read
# Leer sólo un dato:

conexion = sqlite3.connect("database.db")

# Recogemos el código que el usario quiere buscar:
codigo = int(input("Ingrese el código de un artículo:"))

# filtramos por el código y que nos muestre la descripción y precio:
cursor = conexion.execute("SELECT descripcion,precio FROM articulos WHERE␣
 ↪codigo=?", (codigo, ))
```

```
# El método fechone de la clase Cursor retorna una tupla
# con la fila de la tabla que coincide con el código ingresado o retorna 'None':
fila = cursor.fetchone()

if fila != None:
    print(fila)
else:
    print("No existe un artículo con dicho código.")
conexion.close()
```

```
Ingrese el código de un artículo: 2
```

```
('libro', 20.0)
```

[5]:
```
# Recuperar varias filas con una coincidencia en este caso por precio:

conexion = sqlite3.connect("database.db")

# Filtrar los datos por el precio que elija el usuario:
precio = float(input("Ingrese un precio:"))

# Filtrar por dicho valor que sea inferior:
cursor = conexion.execute("SELECT descripcion FROM articulos WHERE precio<?",␣
 ↪(precio, ))

# Llamamos al método 'fetchall' de la clase Cursor y
# nos retorna una lista con todas las filas de la tabla
# que cumplen la condición de tener un precio inferior al ingresado:
filas=cursor.fetchall()

if len(filas)>0:
    for fila in filas:
        print(fila)
else:
    print("No existen artículos con un precio menor al ingresado.")
conexion.close()
```

```
Ingrese un precio: 20
```

```
('boligrafo',)
('lapicero',)
```

[6]:
```
# Update
# Actualizar alguno de los datos buscando por el código

conexion = sqlite3.connect("database.db")

# Buscar el código y añadir el precio por el usuario:
codigo = int(input("Ingrese un código:"))
```

```
precio = float(input("Ingresa un precio:"))

# Actualizar el precio por codigo :
try:
    cursor = conexion.execute("UPDATE articulos SET precio=? WHERE codigo=?;",␣
 ↪(precio, codigo, ))
    # Para que se realize el cambio añadimos .commit
    conexion.commit()
except Exception as e:
    print('Error %s: ' % str(e))

conexion.close()
```

```
Ingrese un código: 3
Ingresa un precio: 1.2
```

[7]:
```
# Delete
# Eliminar un determinado dato

conexion = sqlite3.connect("database.db")

# Eliminar el código que el usuario quiera.
codigo = int(input("Ingrese un código:"))

# Eliminar la fila por el codigo:
try:
    cursor = conexion.execute("DELETE FROM articulos WHERE codigo=?;", (codigo,␣
 ↪))
    #  Para que se realize el cambio añadimos .commit
    conexion.commit()
except Exception as e:
    print('Error %s: ' % str(e))

conexion.close()
```

```
Ingrese un código: 3
```

Creado por:

Isabel Maniega